

Ultimate Resume Builder & AI Coach

An Interview-Ready Technical Architecture Master Guide

Project Overview & Purpose

The **Ultimate Resume Builder & AI Coach** is a premium, client-side rich web application designed to help placement students design resumes, optimize keyword match density, audit layouts for Applicant Tracking Systems (ATS), and practice interviews with an interactive coach.

Why was this project created? Many online builders charge steep monthly subscriptions or store resume data on vulnerable servers. This application empowers students with a completely cost-free (₹0 hosting and compute cost), private, and offline-compatible workspace that uses generative AI models to tailor contents directly inside the user's browser workspace.

Architecture & Tech Stack Specifications

React 19 & Vite

Tailwind CSS (v4)

Lucide Icons

HTML5 Canvas Print Pipeline

RESTful API Integration

Core Tech Decisions:

- **React 19 (Frontend Layer):** Chosen for its declarative component model, state sync behaviors, and quick update cycles. It handles real-time synchronization between input fields in the editor panel and the rendered document canvas.
- **Vite:** Replaced legacy bundling engines like Webpack. It offers instantaneous Hot Module Replacement (HMR) and relies on native ES module serving for swift local coding loops.
- **Tailwind CSS (v4):** Leverages utility classes to design layouts. Avoided custom bloated stylesheets, maintaining rapid visual iterations and smooth responsiveness.
- **Zero-Dependency Print Pipeline:** Instead of using heavy, paid, server-side HTML-to-PDF APIs or client-side canvas-rasterization engines (like `jspdf` or `html2canvas` which blur text), we engineered standard browser print media query stylesheets. Calling `window.print()` yields crisp, searchable, vector-exact PDFs output natively by the browser.



AI Integration & Prompt Engineering Architecture

The application integrates two AI components: the **PrepCoach AI Chatbot** and the **AI Auto-Tailor Engine**. Both are completely client-side.

1. Model Selection & Integration Path

We support **Google Gemini** (`gemini-1.5-flash`) and **Groq** (`llama-3.3-70b-versatile`). Both models are queried via lightweight client-side REST calls (using standard Javascript `fetch`). This design keeps bundle sizes tiny (no external npm dependencies like `@google/generative-ai` or `openai`) and completely eliminates server compute costs.

2. Security & API Keys Strategy

Storing static API keys on web frontends is a severe security vulnerability. To solve this:

- Users configure their own API keys in the chat Settings gear .
- Keys are stored strictly in the client browser's `localStorage` and never sent to a backend database.
- **Local fallback mode:** If no API keys are configured, the app uses a custom built-in NLP matching and rule-based responder. This ensures that recruiters can test the app instantly without registering for keys.

3. Prompt Design & Hallucination Mitigation

To prevent the LLM from making up fake jobs, companies, or universities (hallucinations), we use highly structured system instructions:

```
const prompt = `You are a professional resume writer and recruitment system auditor.  
Here is a candidate's resume in JSON format:  
${JSON.stringify(resume, null, 2)}`
```

```
Here is the target job description:  
${jobDescription}
```

Please optimize the resume JSON to match this job description. Strictly follow these constraints:

1. Keep the name, email, phone, location, and dates EXACTLY the same. Do not make up fake jobs, compa
2. Rewrite the professional summary to specifically align with the key themes in the job description.
3. Tailor experience and project descriptions to emphasize accomplishments, metrics, and technologies

4. Integrate missing skills naturally into the skills lists.
5. Return ONLY a valid JSON string starting with "{" and ending with "}" representing the modified re

How we validate LLM output: Before applying the rewritten resume to React state, we parse the response string. If the LLM returns markdown wrappers (like ````json ... ````), our cleaning regex strips them. We then parse the JSON using `JSON.parse()` and validate that the schema contains key resume fields (like `personalInfo`) to avoid application crashes.

Common Interview Questions & Answers

Q1 "How did you implement the Single Page Fit feature?"

When the user checks the **"Single Page Fit"** option, we inject a custom style block targeting the A4 preview canvas container. This stylesheet applies a fixed height of exactly `297mm` (A4 page height), sets `overflow: hidden` to clip vertical leaks, and scales down paddings, gaps, and line-heights. During system printing, media-queries apply these constraints directly to the page document, compressing all elements onto one clean page.

Q2 "How does the client-side ATS analysis work without a server?"

We wrote a lightweight Natural Language Processing (NLP) helper in pure JavaScript:

1. We filter out a built-in list of 100+ English grammatical **stop-words** (like "and", "or", "the", "duties").
2. We split the job description text using regex, count the frequencies of the remaining words, and identify the top 15 critical keyword competencies.
3. We search for these keywords inside the resume data string. Keywords found are marked as matched, and others are labeled as missing, calculating a 0-100% Match Index.
4. We parse the resume string for passive verbs (e.g. **managed, helped, used**) using a verb map and suggest high-impact action replacements (e.g. **spearheaded, facilitated, leveraged**).

Q3 "What happens if the Gemini/Groq API fails or rate limits?"

Our REST calls are wrapped in a `try...catch` statement. If a request fails (due to network problems, key expiration, or rate limits), the catch block notifies the user via an alert. If it is the first load or if keys are absent, it automatically uses the **Simulated/Hybrid Fallback engine**, ensuring the application remains robust and responsive.

Q4 "How did you implement the 3-second tailoring highlight feedback?"

We declared a `highlightedFields` state holding boolean flags for modified sections. When the tailoring function successfully completes, we set these flags to `true`, which triggers dynamic CSS highlight classes (amber outlines, shadow glow, and bold fonts). Concurrently, we trigger a 3-second `setTimeout` callback that resets the flags back to `false`, ensuring the document transitions back to standard formatting smoothly.